

Trabalho Final INF1022 2026.1

Analizador Sintático ObsAct

Integrantes

- Arthur Barbosa - 2310394
- Gabriel Arruda - 2311723

1. Objetivo

O objetivo do trabalho foi desenvolver um analisador léxico e sintático para a linguagem ObsAct, capaz de receber um programa escrito em ObsAct e gerar um programa equivalente em outra linguagem. A linguagem alvo escolhida foi Python.

O analisador foi implementado com a biblioteca PLY (Python Lex-Yacc), que fornece ferramentas para construção de analisadores léxicos e sintáticos baseados em Lex/Yacc e permite a construção de um parser LALR(1).

2. Arquitetura da Implementação

O projeto foi dividido em cinco módulos principais:

- `lexer.py`: define os tokens da linguagem ObsAct e realiza a análise léxica.
- `parser.py`: define a gramática LALR(1), monta a AST e reconhece programas ObsAct.
- `semantic.py`: realiza validações semânticas, como declaração de dispositivos, observações, duplicatas, limites de tamanho e associações entre dispositivos e observações.
- `codegen.py`: gera o código Python equivalente a partir da AST validada.
- `obsact.py`: driver principal; lê o arquivo `.obsact`, executa `lexer`, `parser` e `semântica`, e escreve o arquivo `.py` gerado.

3. Funcionalidades Implementadas

- Declaração de dispositivos: `dispositivo: {namedevice}` e `dispositivo: {namedevice, observation}`.
- Atribuição via `set observation = VAR`.
- Atribuição via `set observation = ACTEXECUTE`, por exemplo `set estado_ventilador = verificar(ventilador)`.

- Atribuição com contexto de dispositivo: `set {namedevice, observation} = VAR.`
- Valores numéricos inteiros não negativos e valores booleanos TRUE/FALSE, incluindo variações de maiúsculas e minúsculas.
- Condicionais `se OBS então CMDs` e `se OBS então CMDs senão CMDs`.
- Blocos com múltiplos comandos dentro de condicionais e suporte a ifs aninhados.
- Condições compostas com `&&` e operadores `>`, `<`, `>=`, `<=`, `==` e `!=`.
- Ações ligar, desligar e verificar.
- Uso de verificar em condições, como `se verificar(umidificador) == 0 então ...`
- Envio de alerta simples e com variável.
- Envio de alerta sem parênteses, pois alguns exemplos do enunciado usam essa forma.
- Envio broadcast para múltiplos dispositivos.
- Inicialização automática de toda observation para zero.
- Geração das funções de runtime em Python: ligar, desligar, verificar e alerta.
- Laço de repetição enquanto OBS faça { CMDs }, implementado como extensão da linguagem ObsAct. O corpo do laço é delimitado por chaves, e a condição usa o mesmo não terminal OBS dos comandos condicionais.

4. O que foi adicionado à gramática OBSACT?

A gramática do enunciado precisou ser completada para cobrir formas usadas nos exemplos e para tornar implementáveis alguns conceitos que estavam apenas implícitos. Abaixo estão as principais adições, com o ponto do código onde cada uma foi implementada:

- **device_open -> : { | {**: aceita declaração com dois-pontos e sem dois-pontos, como dispositivo: {Monitor} e dispositivo {Monitor}. Código: `parser.py`, função `p_device_open`.
- **full_cmd -> simple_cmd . | simple_cmd**: aceita comandos simples com ou sem ponto final no nível principal. Código: `parser.py`, funções `p_full_cmd_simple` e `p_full_cmd_simple_without_dot`.
- **if_cmds**: define o bloco de comandos de "se ... então CMDS", permitindo múltiplos comandos e ifs aninhados. Código: `parser.py`, funções `p_if_cmds_simple_dot_more`, `p_if_cmds_simple_dot`, `p_if_cmds_simple`, `p_if_cmds_obsact_dot_more`, `p_if_cmds_obsact_dot`, `p_if_cmds_obsact_more` e `p_if_cmds_obsact`.
- **attrib -> set IDENT = actexecute**: permite guardar retorno de ligar, desligar ou verificar em uma variável. Código: `parser.py`, `p_attrib_actexecute`; `semantic.py`, `_validate_cmd`; `codegen.py`, `gen_cmd` e `gen_actexecute_expr`.
- **attrib -> set { IDENT , IDENT } = var**: permite atribuição com dispositivo e observação explícitos. Código: `parser.py`, `p_attrib_device_var`; `semantic.py`, `_validate_cmd` valida se a observação pertence ao dispositivo; `codegen.py`, `gen_cmd`.
- **obs com verificar**: permite condições como `verificar(umidificador) == 0`. Código: `parser.py`, `p_obs_verificar`, `p_obs_verificar_and`, `p_obs_verificar_nospace` e `p_obs_verificar_nospace_and`; `codegen.py`, `gen_obs` e `py_obs`.
- **actexecute -> verificar (IDENT) | verificar IDENT**: explicita verificar como ação executável. Código: `parser.py`, `p_actexecute_verificar_parens` e `p_actexecute_verificar_nospace`; `codegen.py`, `RUNTIME.verificar` e `gen_actexecute_expr`.
- **alert_args**: define mensagem de alerta com/sem parênteses e com/sem observation. Código: `parser.py`, `p_alert_args_msg` e `p_alert_args_var`; `codegen.py`, `gen_alert`.
- **namelist -> IDENT | IDENT , namelist**: define a lista de dispositivos do broadcast. Código: `parser.py`, `p_namelist_one` e `p_namelist_more`; `codegen.py`, `gen_alert` gera uma chamada alerta para cada dispositivo.
- **IDENT com sublinhado em observation**: necessário para nomes como `estado_ventilador`. Código: `lexer.py`, `t_IDENT`; `semantic.py`, `OBSERVATION_RE`.
- **Validações semânticas que não cabem bem na gramática**: dispositivo declarado, tamanho máximo de 100 caracteres, observação declarada e associação dispositivo-observação. Código: `semantic.py`, `validate`, `_validate_cmd`, `_require_device`, `_require_observation` e `_device_name_errors`.
- **loop -> enquanto obs faça { block_cmds }**: adiciona laços de repetição à linguagem ObsAct. Código: `lexer.py` define os tokens enquanto e faça; `parser.py` implementa `p_loop`, `block_cmds` e `block_cmd`; `semantic.py` valida a condição e o corpo do loop; `codegen.py` gera o comando while em Python.
- **Blocos explícitos com chaves**: para evitar ambiguidade em comandos consecutivos, especialmente em casos de ifs seguidos, foi adicionada a possibilidade de delimitar blocos com chaves. Sem um delimitador explícito, o parser pode interpretar comandos seguintes como parte do if anterior, gerando o comportamento conhecido como if "guloso". Com o uso de chaves, o início e o fim do bloco ficam claros na gramática, reduzindo ambiguidades de escopo. Essa ideia também foi usada na implementação do loop enquanto OBS faça { CMDS }, cujo corpo é obrigatoriamente delimitado por chaves.

5. Estado dos Dispositivos

O código Python gerado mantém um dicionário interno com o estado de cada dispositivo declarado. Ao executar ligar namedevice, o estado passa a ser 1. Ao executar desligar namedevice, o estado passa a ser 0. Ao executar

verificar(namedevice), o valor retornado corresponde ao estado atual do dispositivo.

Dessa forma, após executar ligar lampada, a chamada verificar(lampada) retorna 1; após executar desligar lampada, retorna 0.

6. Validações Semânticas

- Todo dispositivo usado deve ter sido declarado.
- Toda observação usada deve ter sido declarada ou criada por atribuição de retorno de ação.
- Dispositivos duplicados são rejeitados.
- namedevice deve conter somente letras e ter no máximo 100 caracteres.
- msg deve ter no máximo 100 caracteres.
- observation deve começar com letra e pode conter letras, números ou sublinhado.
- set {namedevice, observation} valida se a observation pertence ao dispositivo informado. A aceitação de sublinhado em nomes de observação foi incluída para cobrir exemplos do próprio enunciado, como estado_ventilador.
- msg vazia ou contendo apenas espaços é rejeitada.
- VAR é validado como número inteiro não negativo ou booleano.
- ACTEXECUTE é validado para aceitar apenas ligar, desligar e verificar.
- Broadcast sem lista de dispositivos é rejeitado.

7. Gramática Final Utilizada

```
programa      -> devices cmds
devices       -> device devices | device
device        -> dispositivo device_open IDENT }
               | dispositivo device_open IDENT , IDENT }
device_open   -> : { | {
cmds          -> full_cmd cmds | full_cmd
full_cmd      -> obsact . | obsact | simple_cmd . | simple_cmd
simple_cmd     -> attrib | act
attrib        -> set IDENT = var
               | set IDENT = actexecute
               | set { IDENT , IDENT } = var
var -> NUMBER | TRUE | FALSE
obsact        -> se obs entao if_cmds
               | se obs então if_cmds senão if_cmds
if_cmds       -> simple_cmd . if_cmds | simple_cmd . | simple_cmd
               | obsact . if_cmds | obsact . | obsact if_cmds | obsact
  obs -> IDENT oplogic var | IDENT oplogic var && obs |
        verificar ( IDENT ) oplogic var
        | verificar ( IDENT ) oplogic var && obs |
        verificar IDENT oplogic var
        | verificar IDENT oplogic var && obs
oplogic -> > | < | >= | <= | == | != act
-> actexecute
        | enviar alerta alert_args IDENT
        | enviar alerta alert_args para todos : namelist
actexecute -> ligar IDENT | desligar IDENT
               | verificar ( IDENT ) | verificar IDENT
alert_args -> ( STRING ) | ( STRING , IDENT ) | STRING | STRING , IDENT
namelist -> IDENT | IDENT , namelist

full_cmd -> obsact . | obsact | simple_cmd . | simple_cmd | loop . | loop

loop -> enquanto obs faca { block_cmds }

block_cmds -> block_cmd block_cmds | block_cmd

block_cmd -> simple_cmd .
           | simple_cmd
```

```
| obsact .  
| obsact  
| loop .  
| loop
```

8. Alterações em Relação ao Enunciado

- A gramática foi fatorada e organizada para funcionar corretamente com PLY e LALR(1).
- O corpo do if foi implementado com o não-terminal `if_cmds` para permitir múltiplos comandos e ifs aninhados.
- `verificar` foi implementado principalmente na forma `verificar(dev)`, usada nos exemplos, e também aceita `verificar dev`.
- Foi adicionada a funcionalidade extra de laço enquanto OBS faca { CMDS }, traduzida para `while` em Python.
- Foram adicionadas regras `block_cmds` e `block_cmd` para representar blocos explícitos delimitados por chaves no corpo do loop.
- Foram adicionados tratamentos de erro sintático para os principais não terminais da gramática e validações adicionais para os terminais descritos no enunciado.
- Foram adicionados blocos delimitados por chaves para explicitar o escopo de comandos compostos. Essa alteração ajuda a evitar ambiguidades em sequências de ifs e comandos, nas quais o parser poderia associar comandos seguintes ao if anterior. No caso da extensão de loop, as chaves são obrigatórias em enquanto OBS faca { CMDS }.

- Foram aceitos alertas sem parênteses porque alguns exemplos usam enviar alerta "msg" Monitor.
- Foi aceita declaração dispositivo {Monitor}, sem dois-pontos, porque aparece nos exemplos.
- Foram aceitos comandos simples sem ponto final, pois alguns exemplos finais do enunciado aparecem sem ponto.
- Foi aceito sublinhado em observações para nomes como estado_ventilador.

9. Testes Utilizados

Foram usados cinco arquivos .obsact na pasta testes/:

- exemplo1.obsact: declaração de dispositivos, set, condicional simples e ligar.
- exemplo2.obsact: verificar em atribuição, if com múltiplos comandos e if aninhado.
- exemplo3.obsact: if-else, alerta sem parênteses, ligar e desligar.
- exemplo4.obsact: set {dev, obs}, verificar em condição, if aninhado e broadcast.
- exemplo5.obsact: broadcast com variável para múltiplos dispositivos.
- exemplo6_loop.obsact: extensão de laço enquanto OBS faca { CMDS }, com corpo delimitado por chaves. O arquivo gera exemplo6_loop.py e, ao executar o Python gerado, liga a lâmpada, verifica seu estado e envia o alerta final.
- exemplo7_loop_enquanto.obsact: segundo teste positivo da extensão de laço enquanto, confirmando que o loop é reconhecido pelo parser, validado semanticamente e traduzido para while em Python. O arquivo gera exemplo7_loop_enquanto.py e executa corretamente.
- exemplo8_if.obsact: caso com condicionais consecutivos. O arquivo gera exemplo8_if.py e sua execução termina sem erro de Python.

Também foram testados casos adicionais: comandos sem ponto final, verificar após ligar, rejeição de namedevice com mais de 100 caracteres e rejeição de set {dev, obs} quando a observação não pertence ao dispositivo. Nos testes negativos, o comportamento esperado é o compilador recusar o programa e não gerar arquivo .py. Isso ocorre nos exemplos 9, 11, 12, 13, 14 e 15. O exemplo 10 gera .py, mas sua execução gera UnboundLocalError por colisão de nome com uma função interna do runtime.

Ao todo, foram utilizados 15 arquivos de teste na pasta testes/. Desses, 9 testes representam programas ObsAct aceitos pelo compilador e, portanto, geram código Python: exemplos 1, 2, 3, 4, 5, 6, 7, 8 e 10. Os exemplos 1 a 8 também foram executados em Python para verificar o comportamento do código gerado. O exemplo 10 foi mantido como caso extra de colisão de nome com função interna do runtime: ele gera o arquivo Python, mas sua execução resulta em UnboundLocalError, pois a observation inicializar_dispositivos colide com a função interna de inicialização.

Os outros 6 testes são testes negativos, isto é, programas ObsAct propositalmente inválidos que devem ser rejeitados pelo analisador. Por isso, os exemplos 9, 11, 12, 13, 14 e 15 não geram arquivo .py. Eles verificam, respectivamente: mensagem vazia, if com corpo vazio, uso de dispositivo como observation, namedevice inválido, número negativo e broadcast sem lista de dispositivos. Nesses casos, o comportamento correto é exibir erro léxico, sintático ou semântico e interromper a geração de código.

10. Como Executar

```
pip install ply
python obsact.py entrada.obsact saida.py
python saida.py
```

Exemplo:

```
python obsact.py testes/exemplo1.obsact testes/exemplo1.py
python testes/exemplo1.py
```

Ou rodar o script de teste:

```
python gerar_testes.py
python gerar_testes.py --run
```

Na nossa implementação, deixamos os casos de teste dentro de uma pasta “testes”. Todas as funcionalidades exigidas pelo enunciado foram implementadas e testadas: análise léxica, análise sintática LALR(1), geração de código Python, declaração de dispositivos, atribuições, ações, verificação de estado, alertas simples e com variável,

broadcast, condicionais com múltiplos comandos, if-else, ifs aninhados e validações semânticas.

Também foi criado um script/comando de testes para automatizar a geração e execução dos arquivos. Esse procedimento percorre os arquivos .obsact da pasta testes/, chama o obsact.py para gerar os respectivos arquivos .py e, nos casos em que a geração é bem-sucedida, executa o Python gerado. Isso facilita a verificação conjunta dos exemplos aceitos e dos testes negativos, mostrando no terminal quais arquivos foram gerados corretamente e quais foram rejeitados pelo analisador.

EXEMPLO DE EXECUÇÃO:

```
PS C:\Users\barbosaarthur\Desktop\Trabalho anal\Trabalho-INF1022> python gerar_testes.py
[OK ] exemplo1.obsact
[OK ] exemplo10_colisao_inicializar_dispositivos.obsact
[REJEI] exemplo11_entao_ponto.obsact
        ERRO SINTATICO: token DOT('.') linha 4
        ERRO SINTATICO: OBSACT invalido; esperado se OBS entao CMD ou se OBS entao { CMDS }
        Falha na analise.
[REJEI] exemplo12_umidade.obsact
        ERRO SEMANTICO: observation 'umidade' usada em condicao nao foi declarada em nenhum dispositivo
[REJEI] exemplo13_namedevice_invalido.obsact
        ERRO SEMANTICO: namedevice 'Sensor1' invalido em declaracao; use somente letras
        ERRO SEMANTICO: namedevice 'Sensor1' invalido em acao; use somente letras
[REJEI] exemplo14_num_negativo.obsact
        ERRO LEXICO: caractere inesperado '-' linha 2
        Falha na analise.
[REJEI] exemplo15_broadcast_sem_lista.obsact
        ERRO SINTATICO: token DOT('.') linha 2
        ERRO SINTATICO: namelist invalida; esperado pelo menos um namedevice
        Falha na analise.
[OK ] exemplo2.obsact
[OK ] exemplo3.obsact
[OK ] exemplo4.obsact
[OK ] exemplo5.obsact
[OK ] exemplo6_loop.obsact
[OK ] exemplo7_loop_enquanto.obsact
[OK ] exemplo8_if.obsact
[REJEI] exemplo9_mensagem_vazia.obsact
        ERRO SEMANTICO: msg nao pode ser vazia

-----
compilaram: 9   rejeitados(esperado): 6   falhas: 0
Tudo conforme esperado.
```


